Lab 10. Running Containers at Scale with Kubernetes

By the end of this lab exercise, you should be able to:

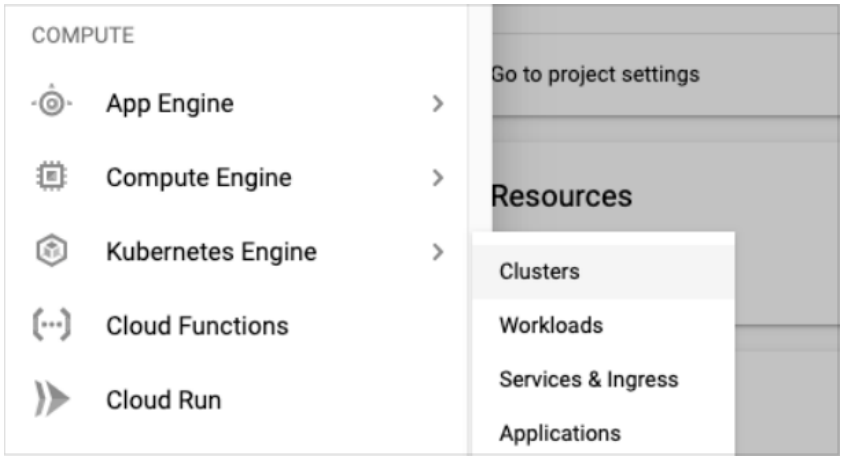
- Set up a managed Kubernetes environment with Google Cloud
- Set up a **`kubectl`** client, connect and work with the Kubernetes cluster
- Deploy applications to the Kubernetes cluster and expose them as services

Set Up Kubernetes Cluster with GCP

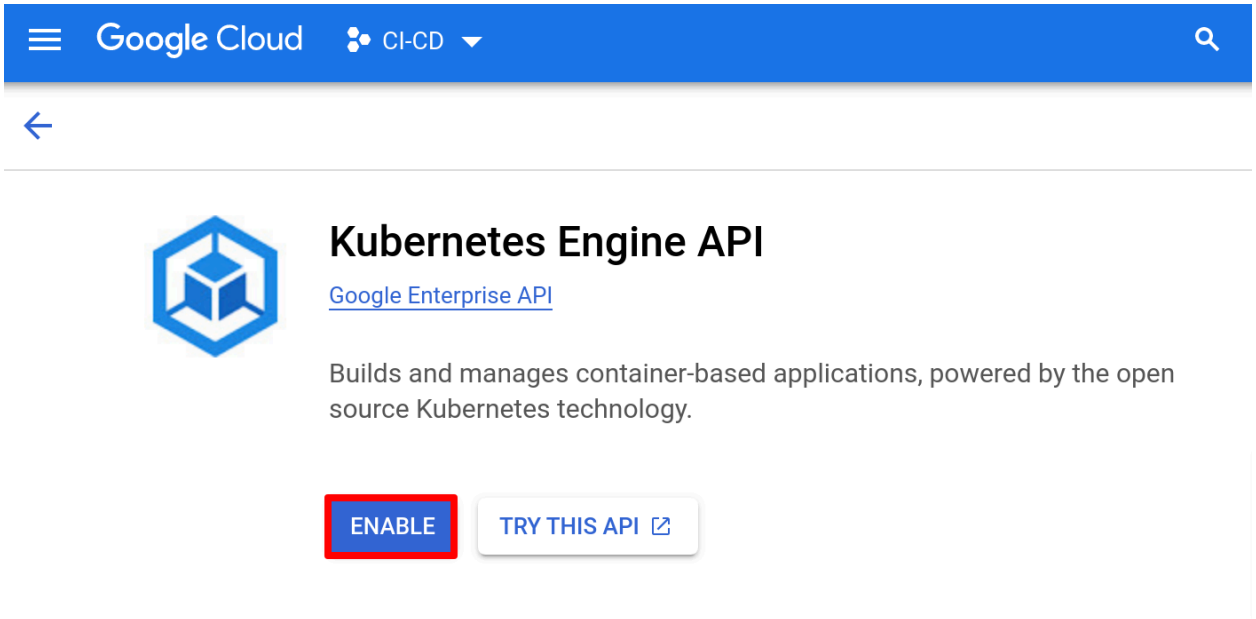
Set up a Google Cloud account by browsing to https://cloud.google.com/free.

Once setup, login to your account and browse to the cloud console by visiting https://console.cloud.google.com/.

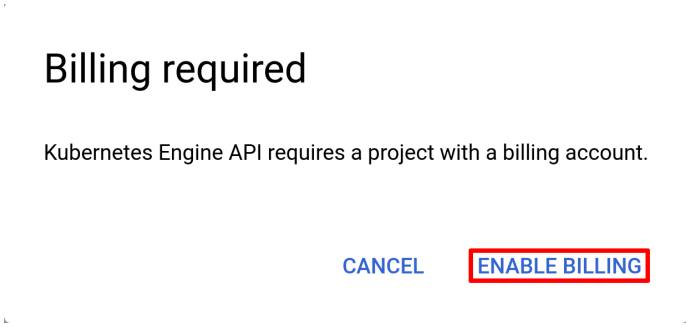
From **Navigation** select **View All Products > Compute > Kubernetes Engine > Clusters**.

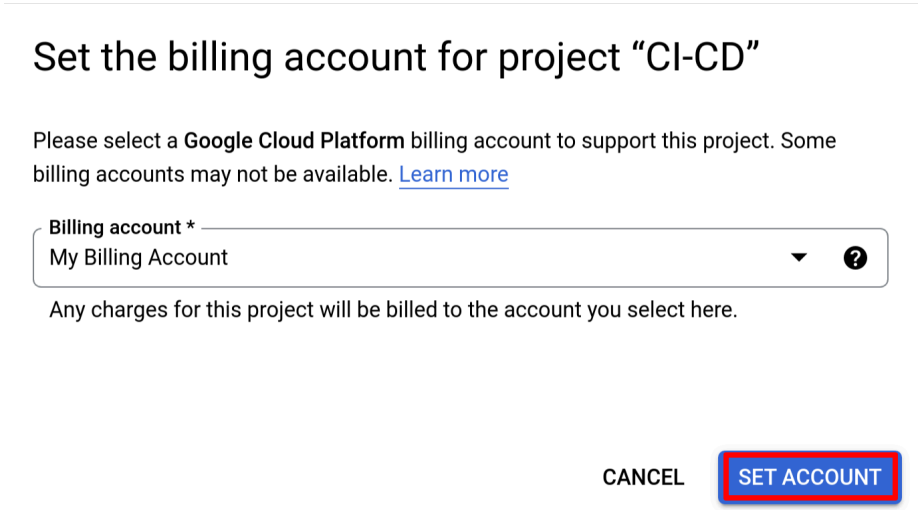


If it asks you to enable Kubernetes Engine API, do so by clicking on the **Enable** button.



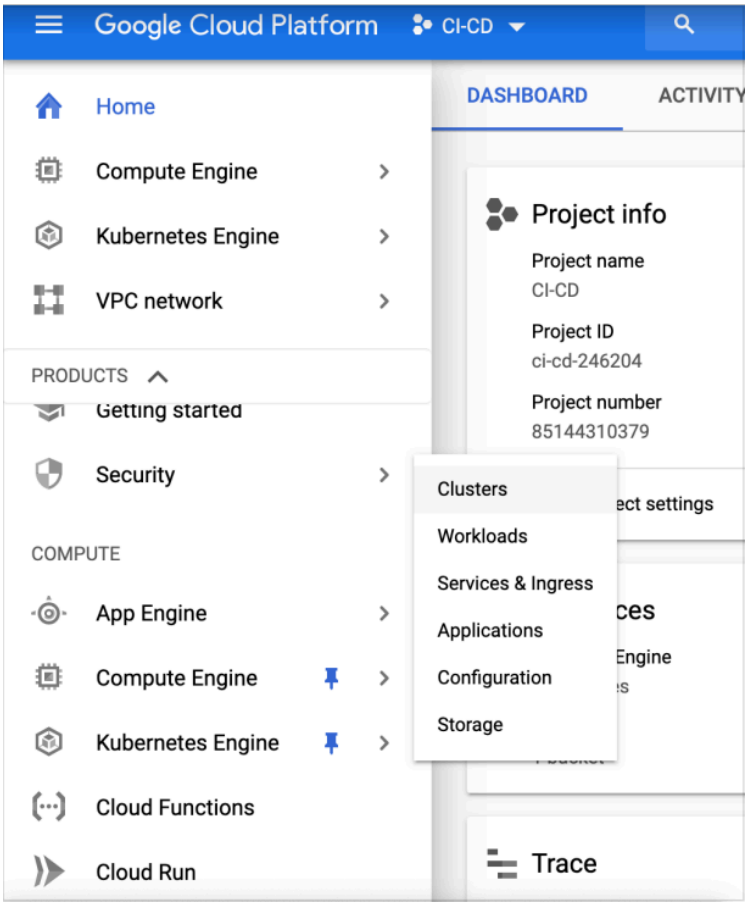
You may also be asked to set up a Billing Account, which you should set up.

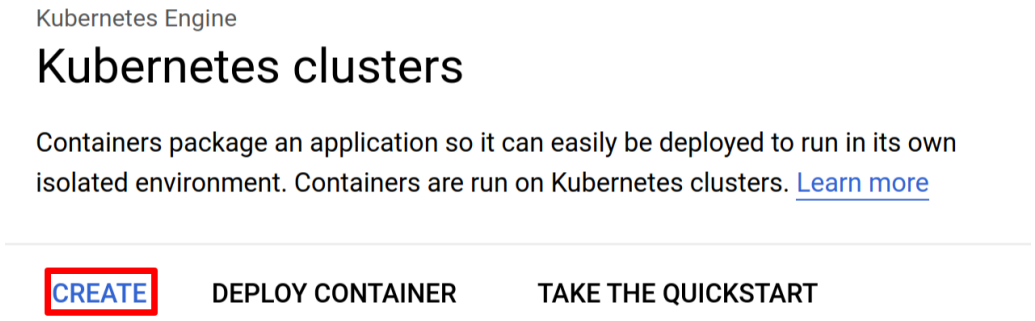




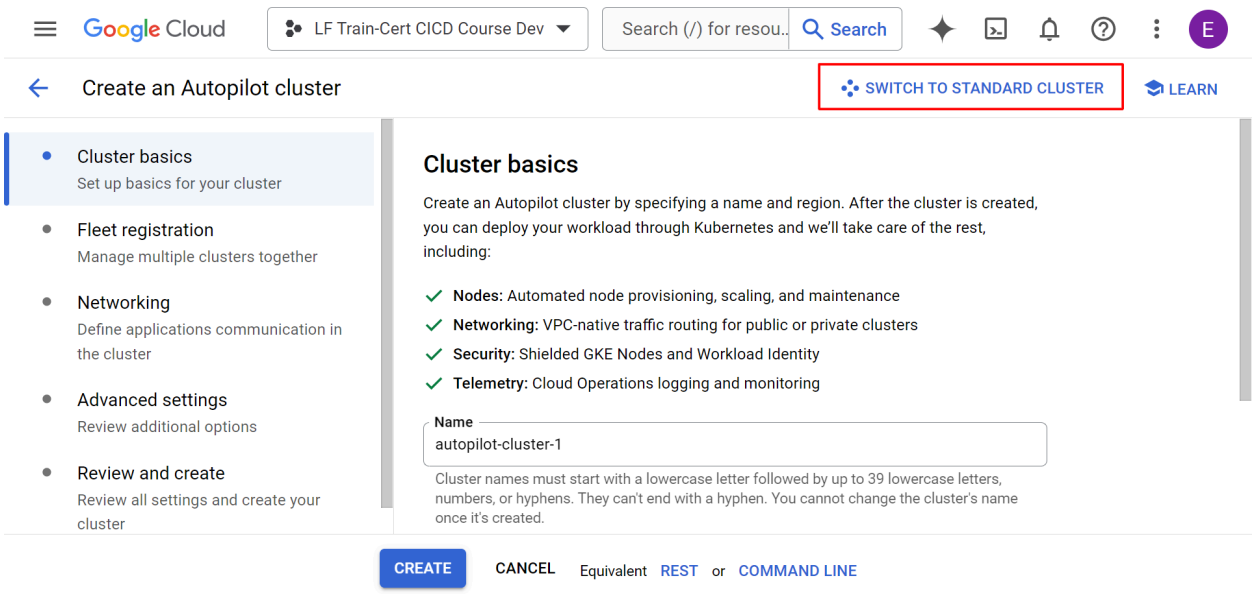
Be aware that if you are signing up for a Free Trial with some free credits, enabling a billing account does not automatically charge you. When you run out of free credits, you will be given some time to allow Google Cloud to charge your credit card. Your resources may be deleted if you run out of free credits and do not allow Google Cloud to charge your credit card.

Once you have completed these steps, go to **Google Cloud Console**, select **Kubernetes Engine > Clusters** and you will arrive at the following page, ready to launch a cluster.

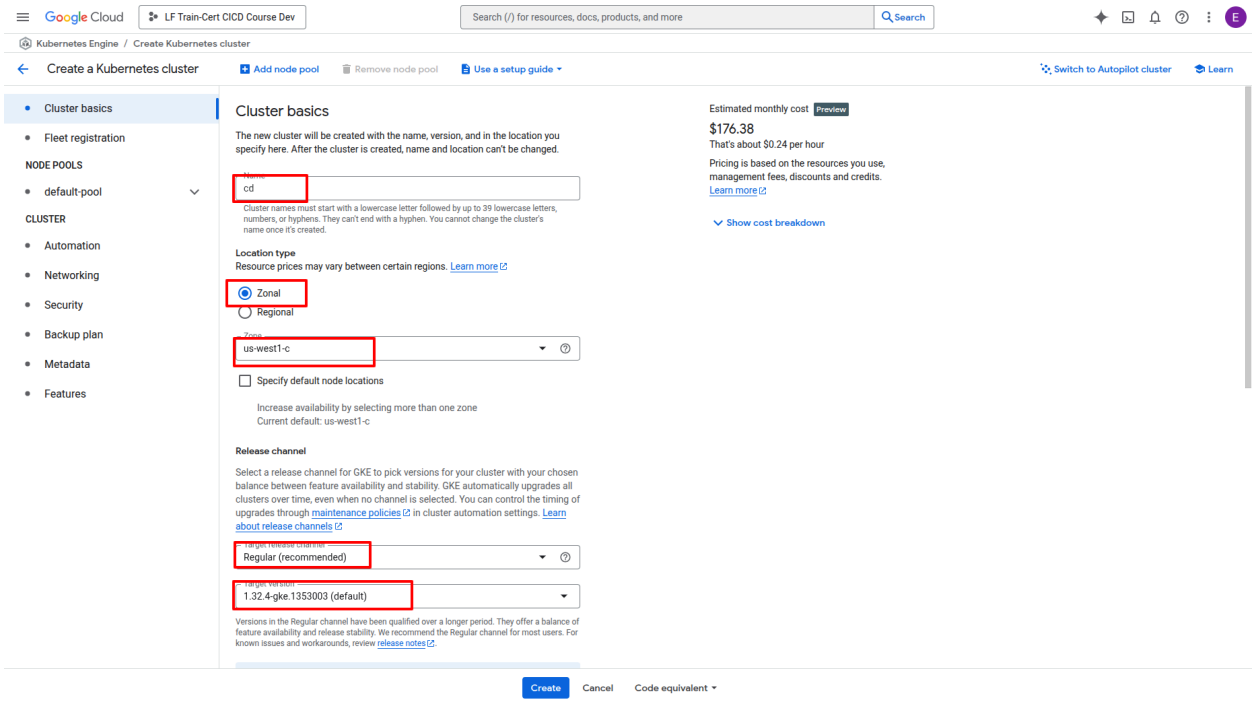




Click on **Create Cluster**, choose **Standard.**



This opens up a cluster configuration page similar to below. Provide the name for the cluster, e.g. **cd**, and select **Regular Channel** for control plane version. Do note, the actual version that you see may differ from what is shown in the screenshots.



Click on **Create** and wait for the cluster to be ready.

Connecting to Your Cluster

There are two ways to connect to your cluster. Either way will end up with you interacting with your cluster via a terminal in which you will run the commands given.

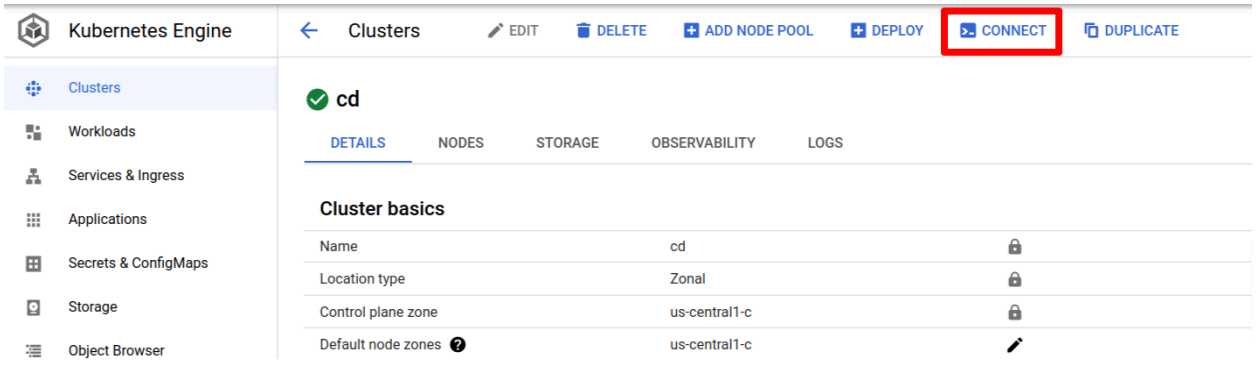
The first and easier way to connect to your cluster is via the **RUN IN CLOUD SHELL** option. The second is to manually connect to your cluster via your own machine.

We will go over both but it is suggested you use the easier method.

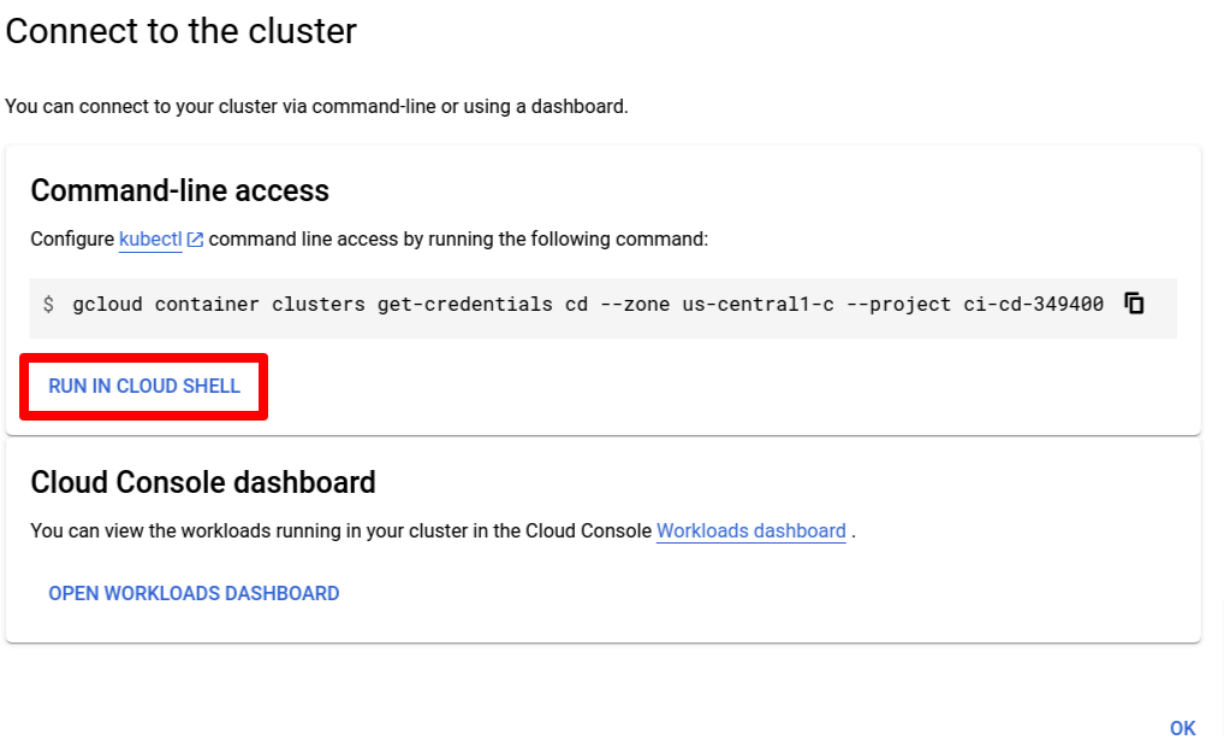
Connect to Your Cluster Via **RUN IN CLOUD SHELL**

Navigate to **Kubernetes Engine > Clusters > click on your cluster name**, as follows:

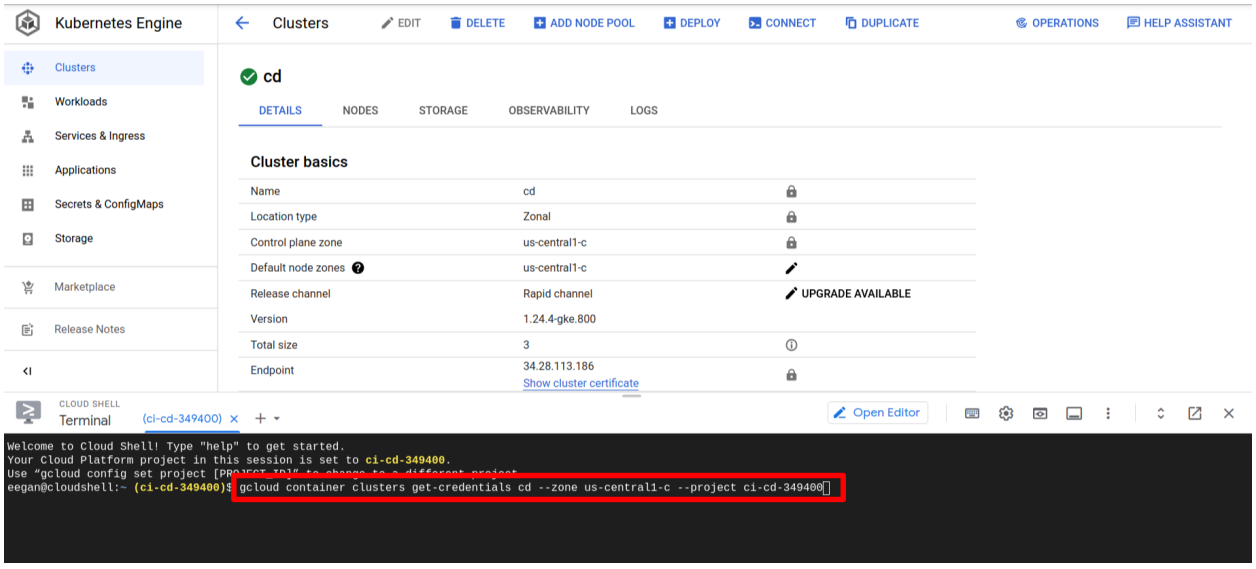
In the horizontal menu across the top of the screen you will see the **CONNECT** option. Click it:



A pop up window will appear. Click **RUN IN CLOUD SHELL**:



A terminal will appear with the **gcloud** command to connect to your cluster pre-populated.



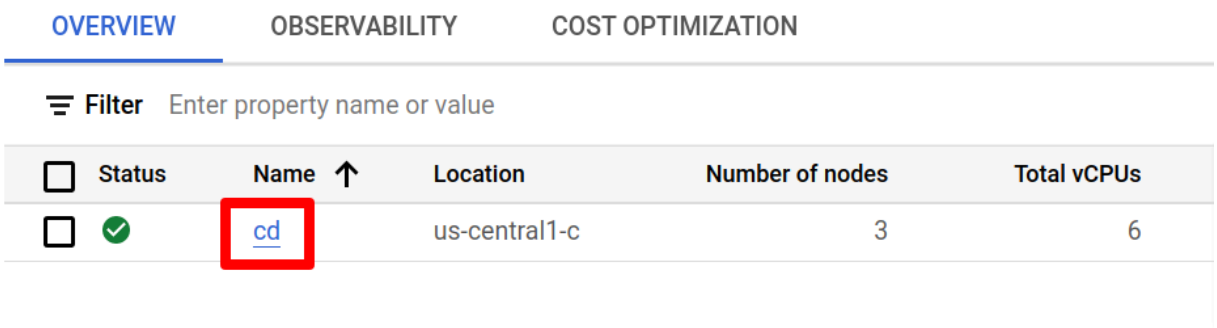
Press **Enter** and you will be connected to your cluster via a terminal.

Manual Connection to Your Cluster

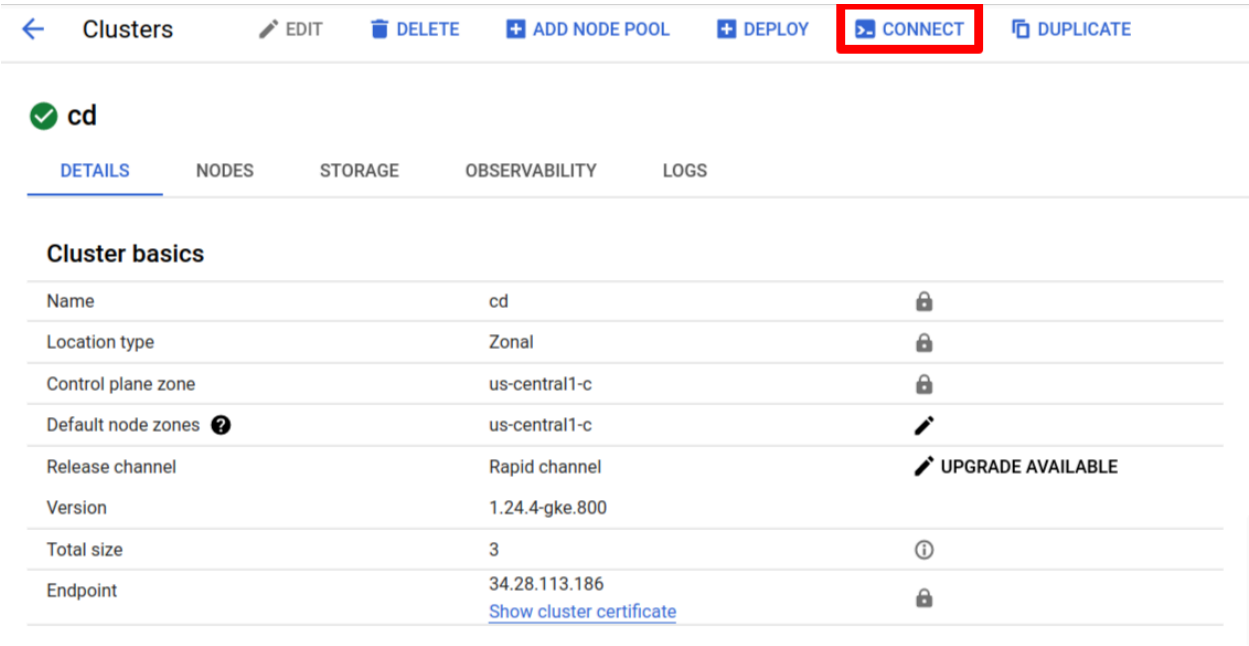
To connect to your cluster via your own machine, use the following:

- Read the Google SDK installation guide.
- Install Google SDK with interactive option.
- Install kubectl by referring to the documentation relevant to your OS.

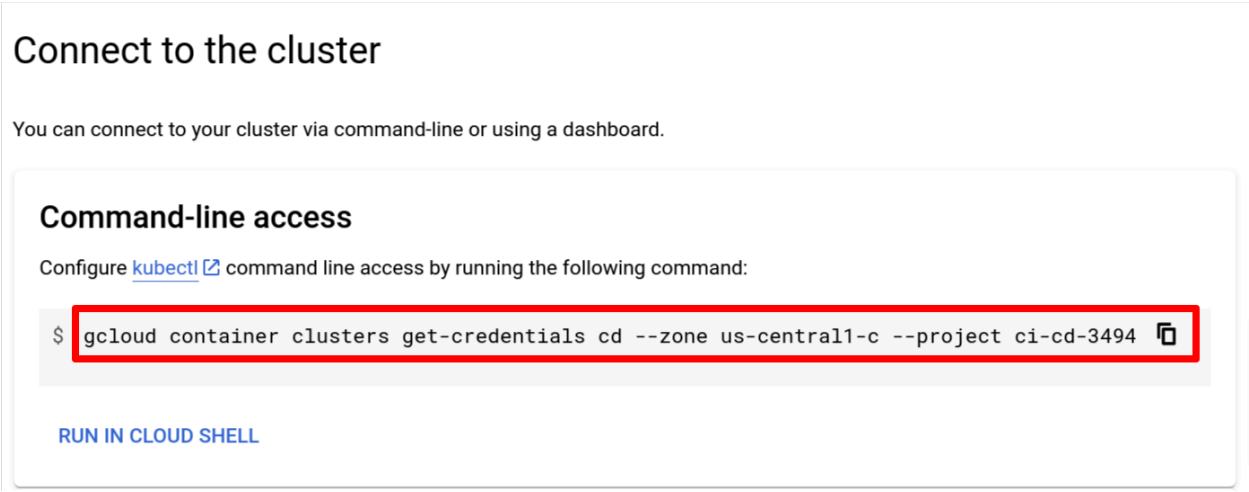
Navigate to **Kubernetes Engine > Clusters** and click on the name of your cluster.



Click on **CONNECT.** If you don't see it, you are likely zoomed in too far in your browser settings and it will be under the vertical three-dot menu on the right.



A popup window should appear. Copy the **`Connect to the cluster`** command and run it on the host where you have installed **`kubectl`**.



Verify **`kubectl`** configuration using the following commands:

```
kubectl version -o yaml
```

```
clientVersion:
  buildDate: "2025-05-15T20:09:34Z"
  compiler: gc
  gitCommit: 4cb5f0764b8d5f425645c6f433394fede34a8a26
  gitTreeState: clean
```

```
  gitVersion: v1.32.4-dispatcher
  goVersion: go1.23.8
  major: "1"
  minor: 32+
  platform: linux/amd64
kustomizeVersion: v5.5.0
serverVersion:
  buildDate: "2025-05-19T04:19:46Z"
  compiler: gc
  gitCommit: d05b573072d7732178c3b9a376809c1d945f669f
  gitTreeState: clean
  gitVersion: v1.32.4-gke.1353003
  goVersion: go1.23.8 X:boringcrypto
  major: "1"
  minor: "32"
  platform: linux/amd64
```

```
kubectl get nodes
```

When you run `kubectl get nodes` you should see three nodes listed with **Ready** status.

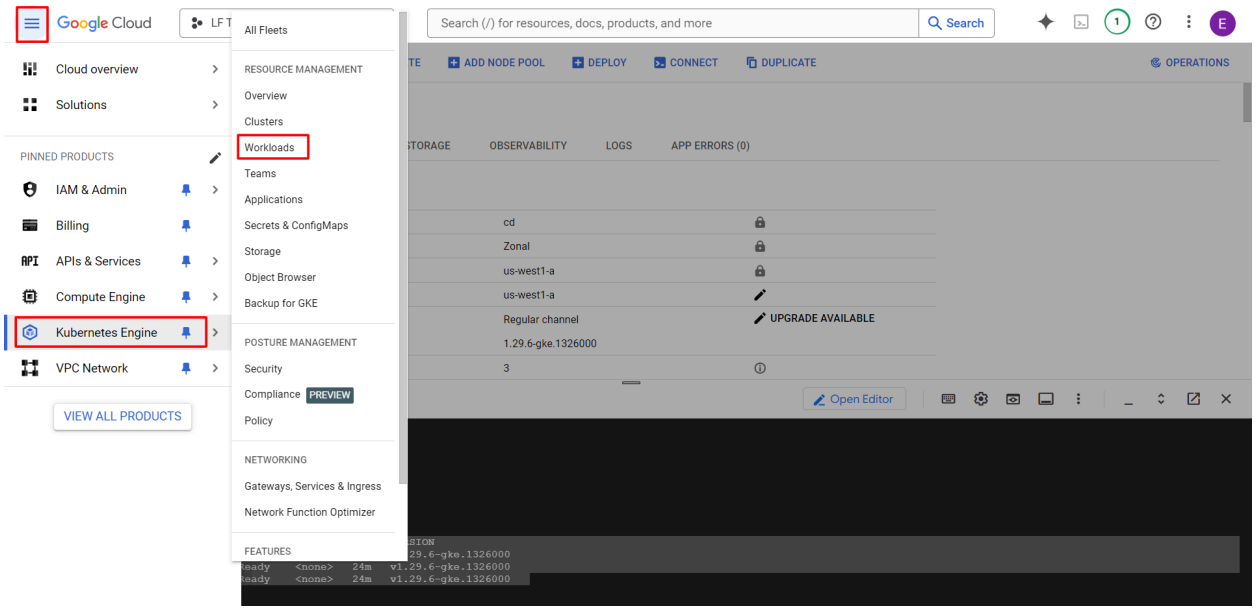
Sample output:

```
NAME                                  STATUS   ROLES    AGE   VERSION
gke-cd-default-pool-4c6b87d8-39cg   Ready    <none>   26m   v1.32.4-gke.1353003
gke-cd-default-pool-4c6b87d8-9rbn   Ready    <none>   26m   v1.32.4-gke.1353003
gke-cd-default-pool-4c6b87d8-pwg0   Ready    <none>   26m   v1.32.4-gke.1353003
```

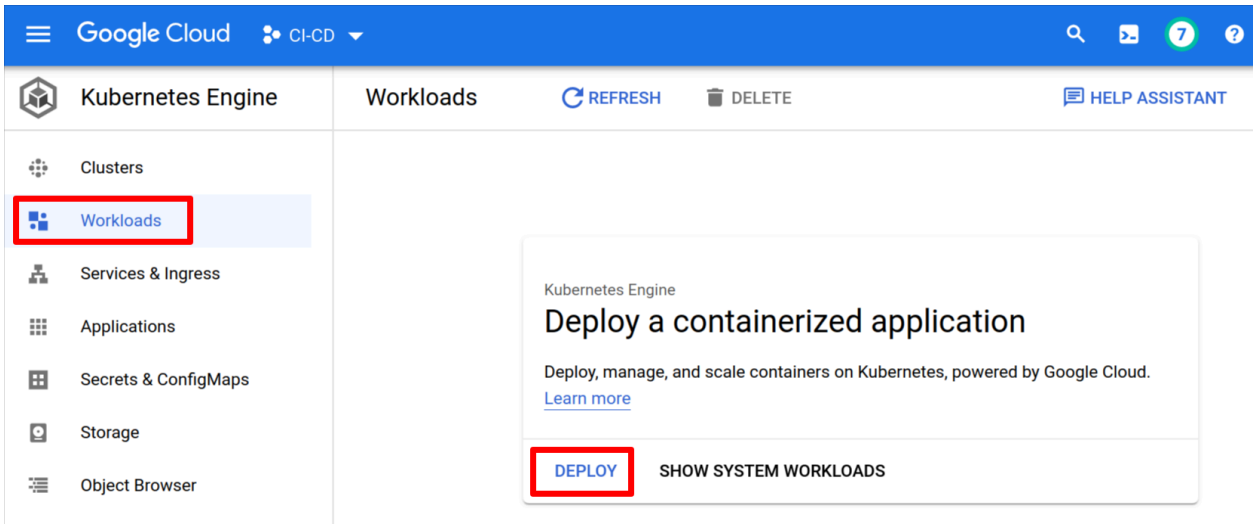
This validates that the cluster is ready and operational.

Deploy the Frontend Vote App with Kubernetes

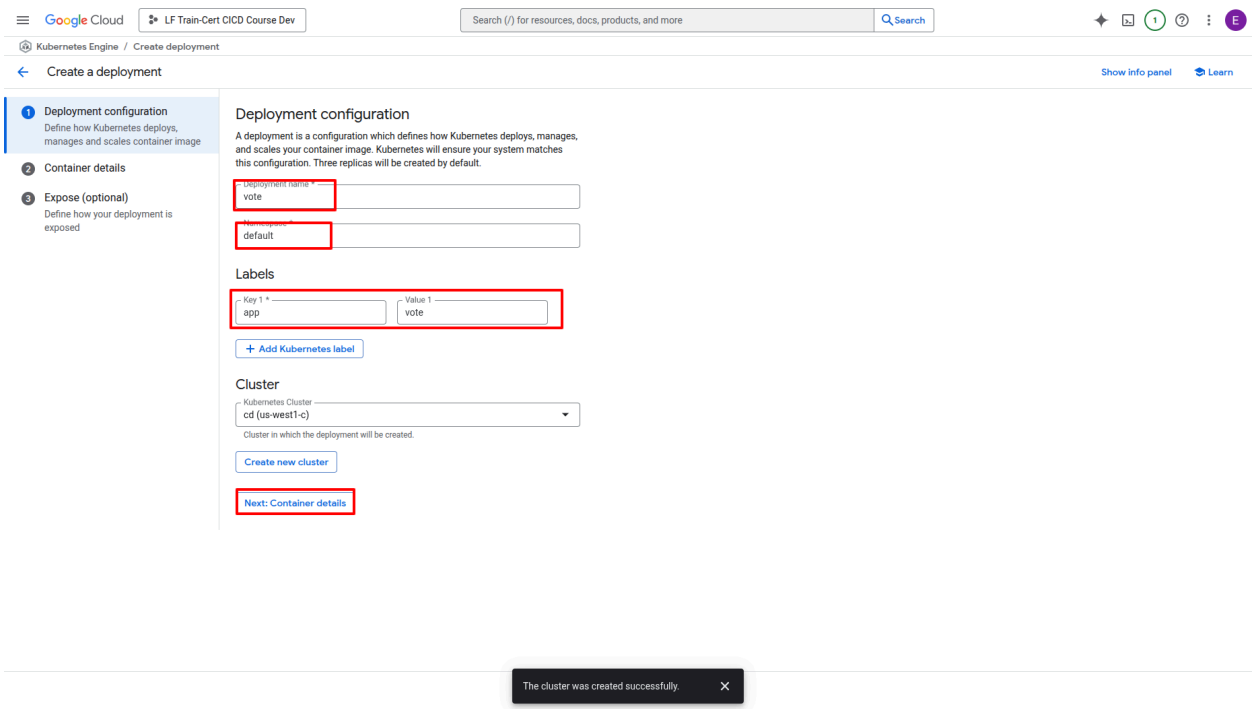
Navigate to Google Kubernetes Engine's **Workloads:**



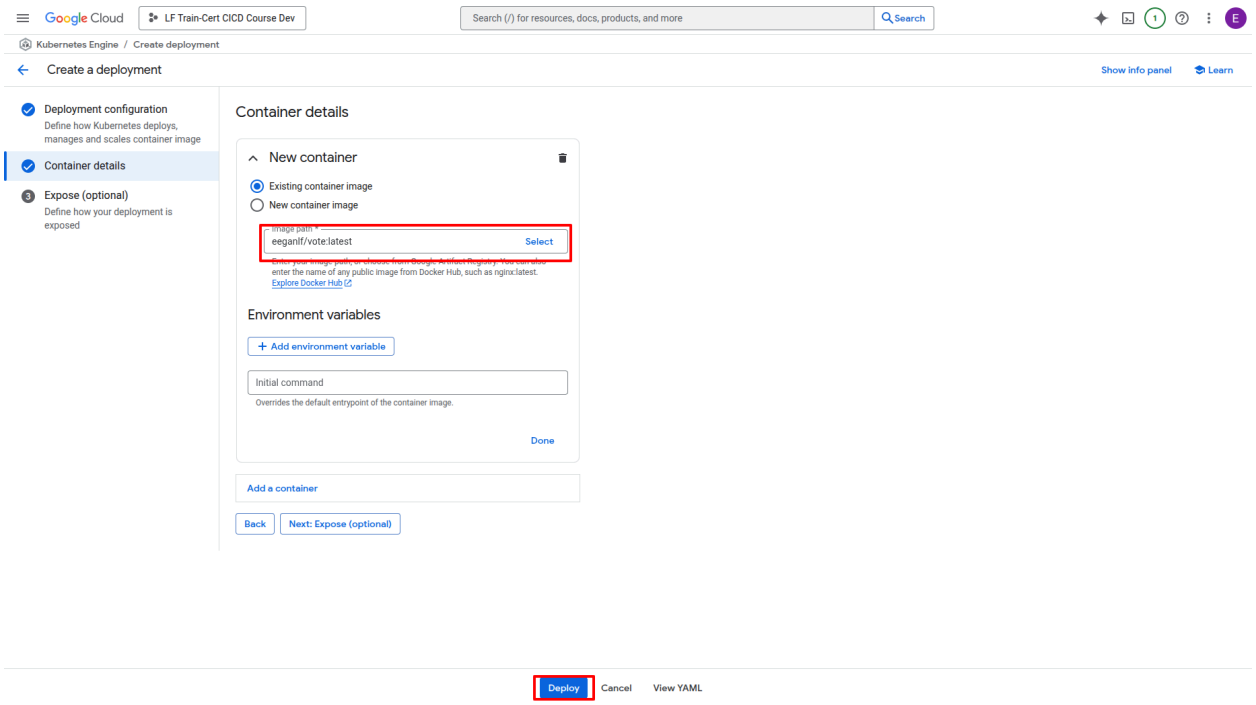
Click **Create Deployment**.



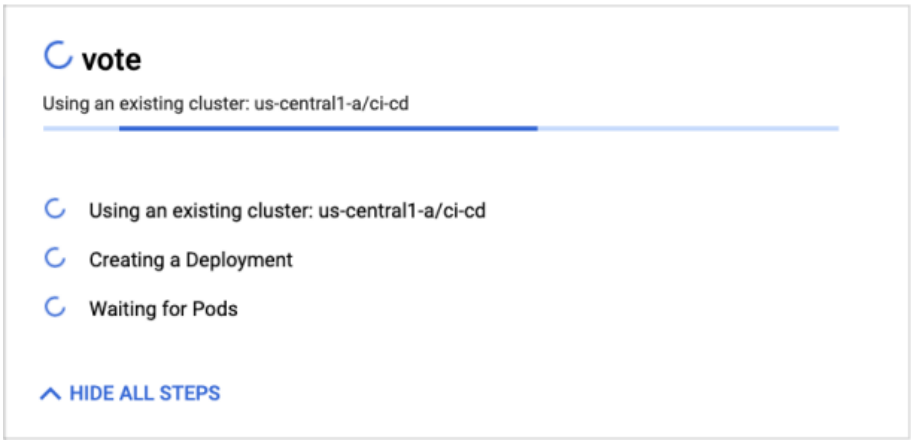
Provide the configurations to deploy the frontend **`vote`** app as follows and click **Next** to access the container details page.

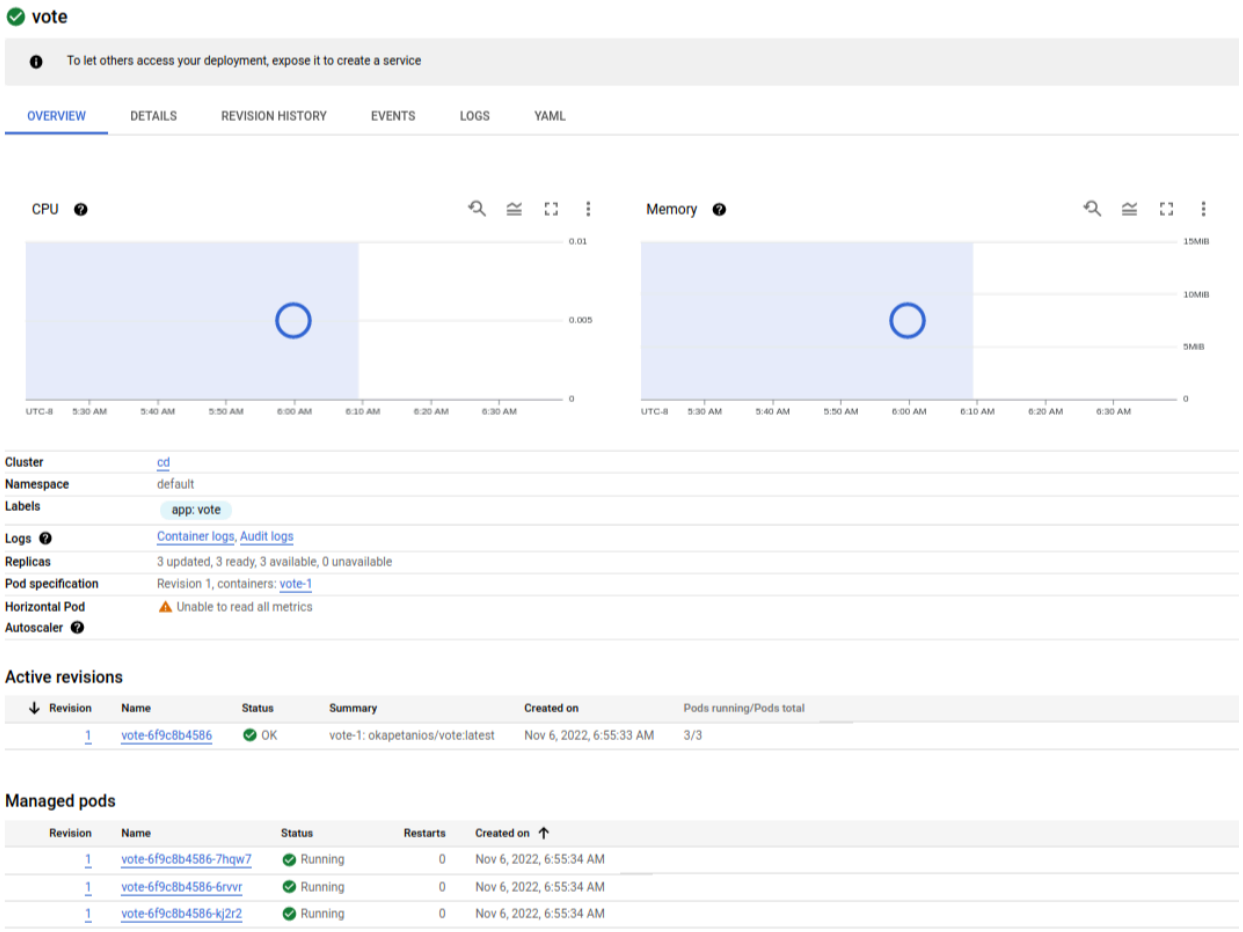


We will use our existing container image: **`DockerHubUserID/vote`**. NOTE: Replace the image with your actual image tag.



Click **Deploy**.





Your app is now deployed on the cluster.

Pod Operations

From the host where **`kubectl`** is configured, you can explore pod operations by running the following commands:

```
kubectl get pods
```

```
kubectl get pods -o wide
```

***NOTE**: Replace **`vote-xxxx`** with the actual pod names displayed under **`kubectl get pods`**.*

```
kubectl describe pods vote-xxxx
```

```
kubectl logs vote-xxxx
```

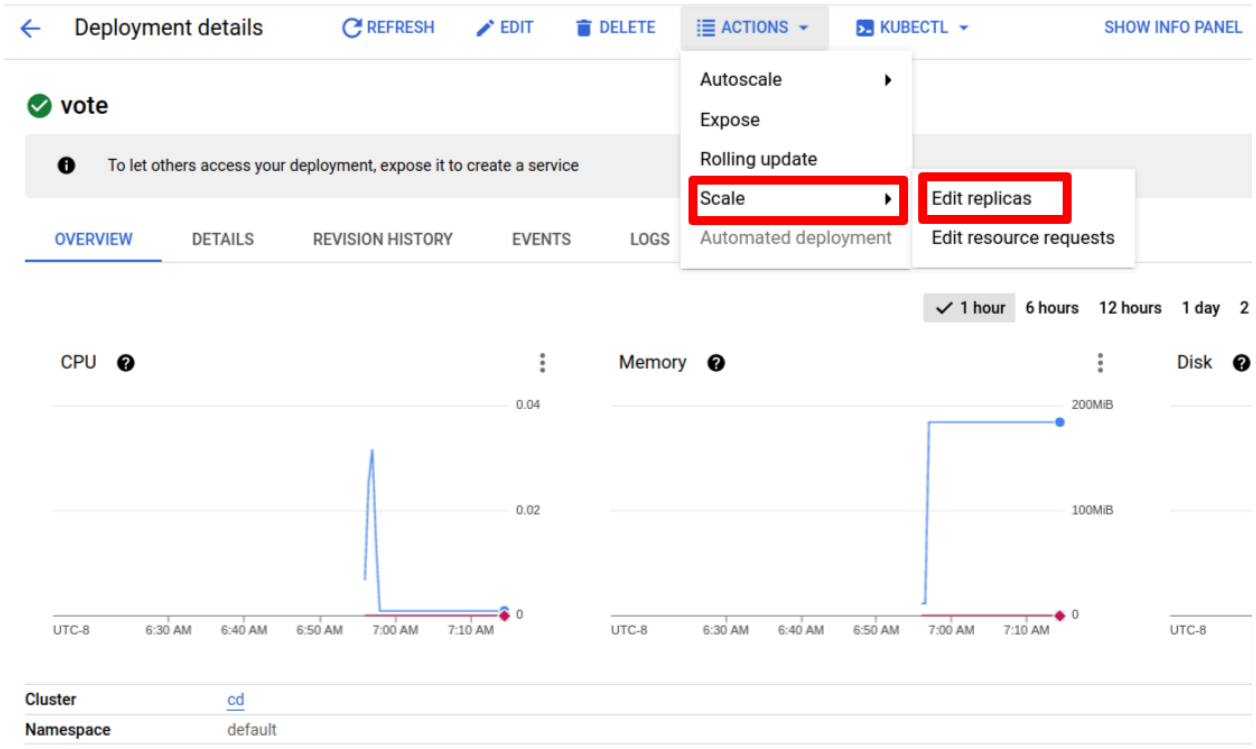
```
kubectl exec vote-xxxx -- ps

kubectl exec -it vote-xxxx -- sh
```

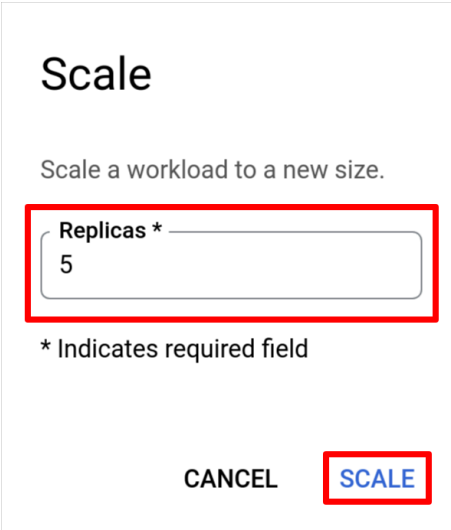
Use **CTRL + d** to exit from the container's shell.

Scalability

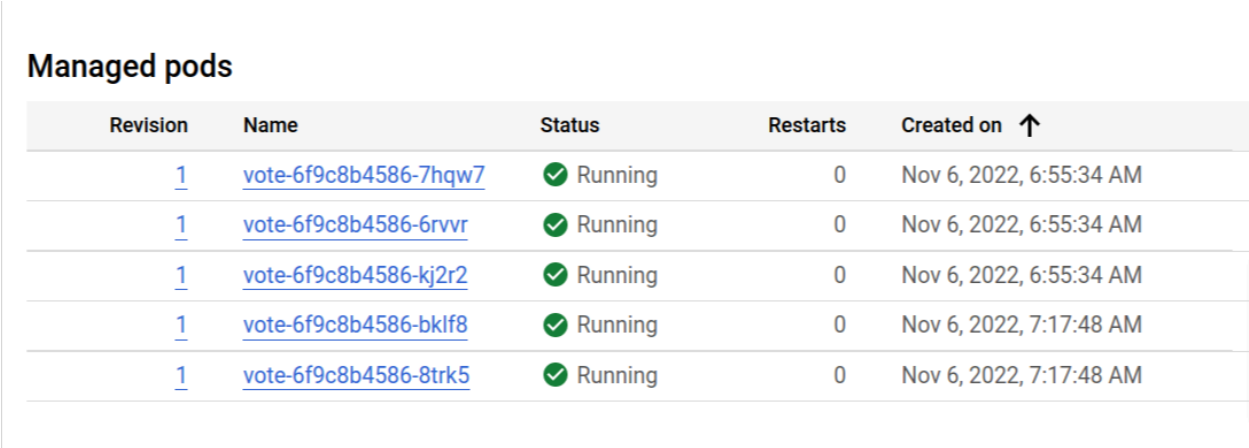
Scaling Manually

Click the **ACTIONS** menu and select **Scale > Edit replicas.**





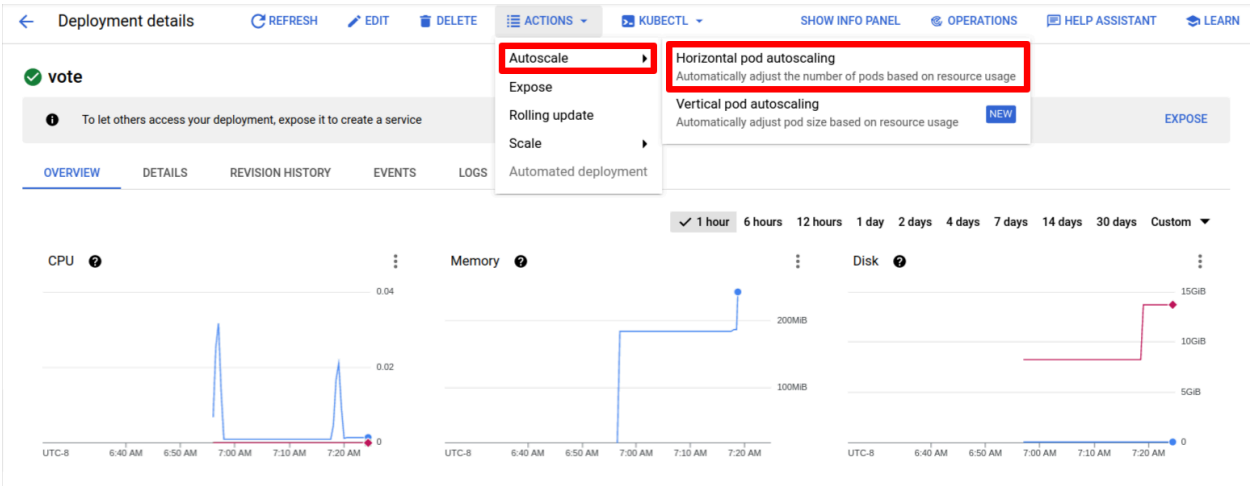
Under the **Managed pods** section, you will now see that 5 pods have been deployed.



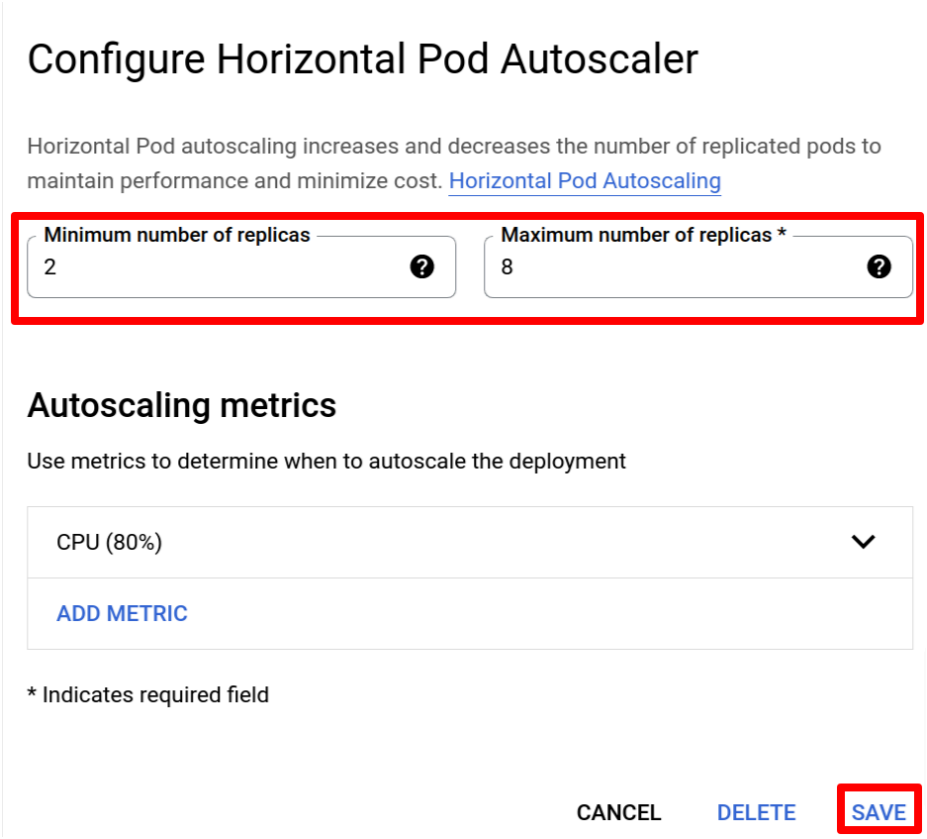
You have manually scaled your application using Kubernetes with Google's Kubernetes Engine.

Autoscaling

You will now configure Kubernetes to handle the scaling for your cluster automatically. Once again, navigate to **ACTIONS**. This time select **Autoscale > Horizontal pod autoscaling** from the dropdown, as follows:



Give your deployment a minimum of 2 replicas and a maximum of 8, as follows:



You will now test that your autoscaling works by attempting to delete pods.

Availability

Try deleting a few pods after listing them and observe what happens. You should see new pods created automatically.

```
$ kubectl get pods

NAME                    READY   STATUS    RESTARTS   AGE
vote-799cf486bf-ctvww   1/1     Running   0          4s
vote-799cf486bf-jmr5b   1/1     Running   0          8s
vote-799cf486bf-jnwbs   1/1     Running   0          8s
vote-799cf486bf-p5wzc   1/1     Running   0          8s
```

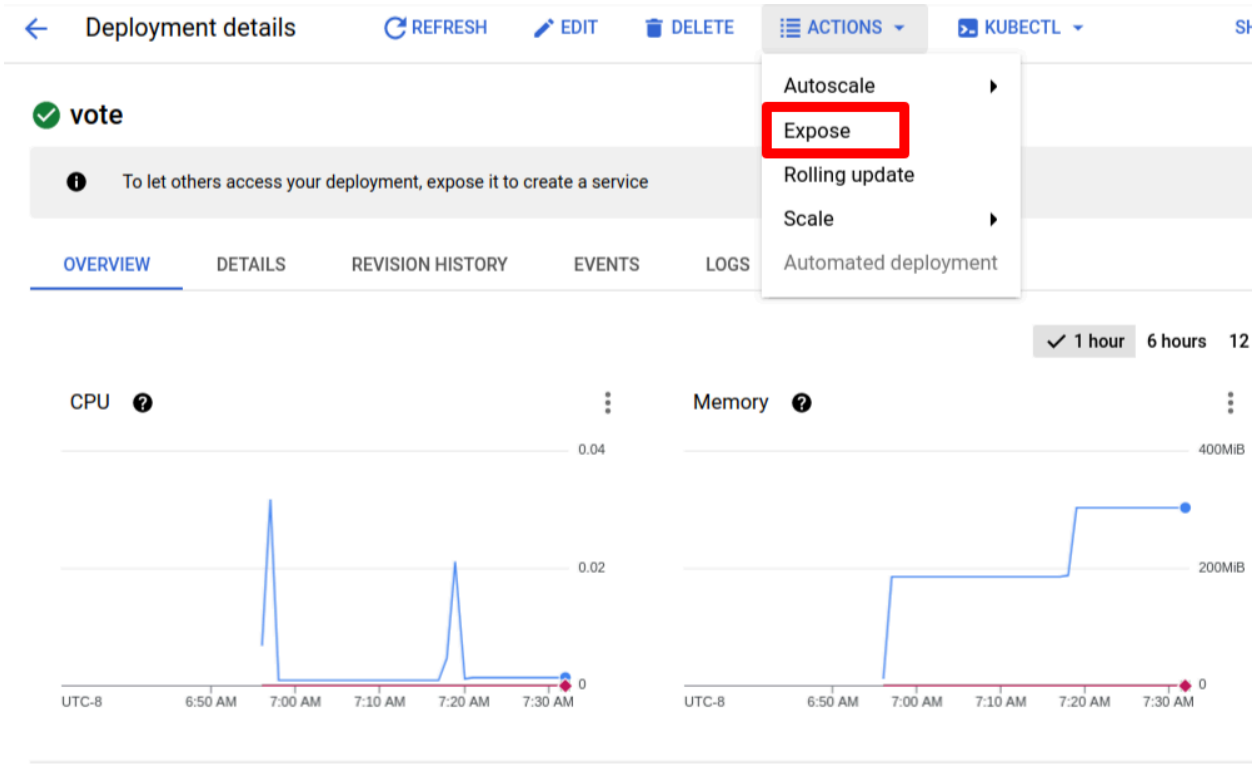
***NOTE**: Replace the following **vote-xxxx-xxxx** pod names with your own.*

```
$ kubectl delete pods vote-799cf486bf-jnwbs vote-799cf486bf-p5wzc
pod "vote-799cf486bf-jnwbs" deleted
pod "vote-799cf486bf-p5wzc" deleted

$ kubectl get pods

NAME                    READY   STATUS    RESTARTS   AGE
vote-799cf486bf-bw4bf   1/1     Running   0          14s
vote-799cf486bf-ctvww   1/1     Running   0          34s
vote-799cf486bf-d7dzq   1/1     Running   0          14s
vote-799cf486bf-jmr5b   1/1     Running   0          38s
```

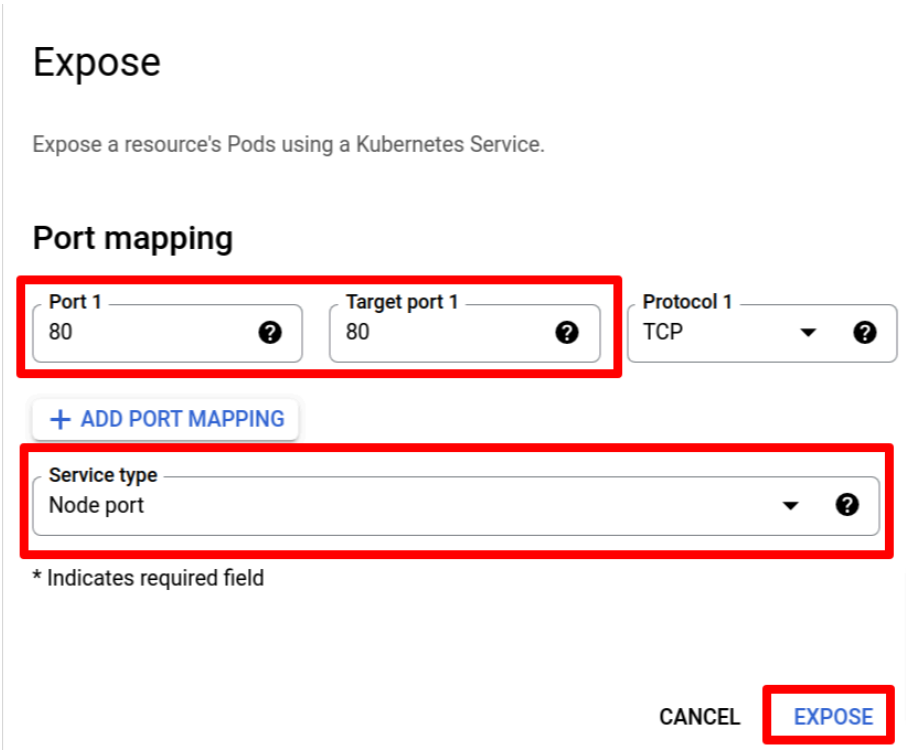
Kubernetes has created new pods to replace the ones you deleted. Kubernetes always maintains the desired scale that you provide and constantly monitors and takes action if the desired count changes. This is an example of availability, fault tolerance, and resilience.

Publishing and Load Balancing with Services

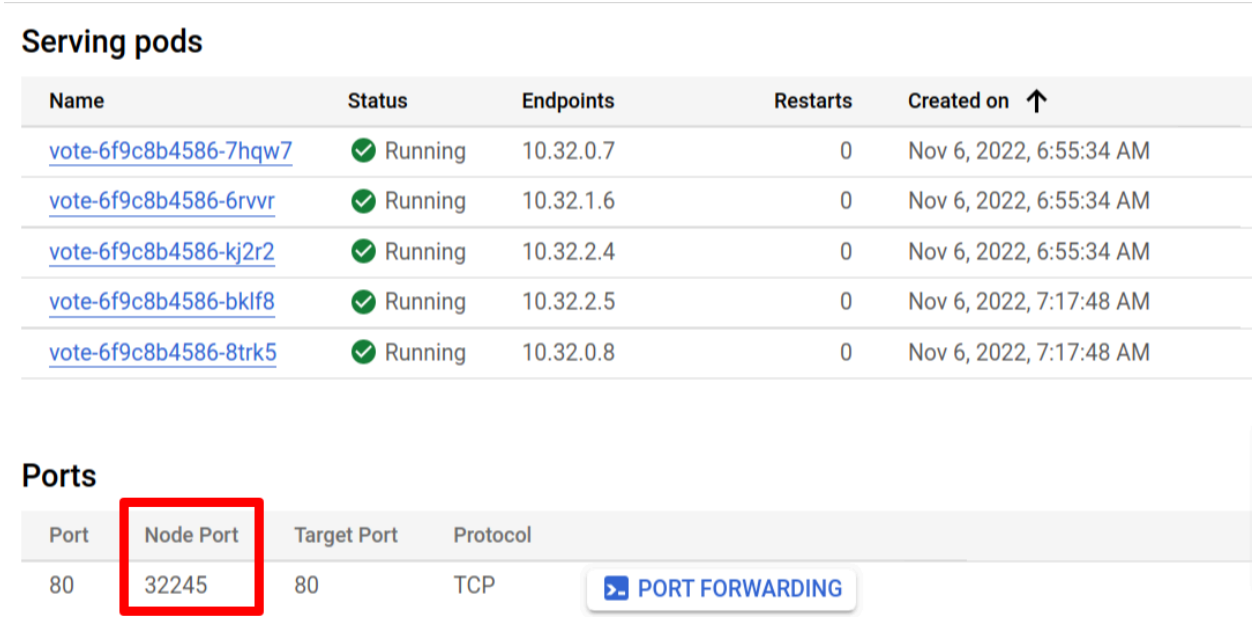
Browse to **Workloads > vote > Actions** and then select **Expose**.



Provide values for “Port” and “Target Port” as 80 and then choose **NodePort** from the **Service type** dropdown. This is an important step in order to access this application from outside the cluster.



Once created, browse to **Kubernetes Engine > Gateways, Services & Ingress > Services > vote-xxxx.** Scroll to the **Ports** section and find the NodePort associated with it.

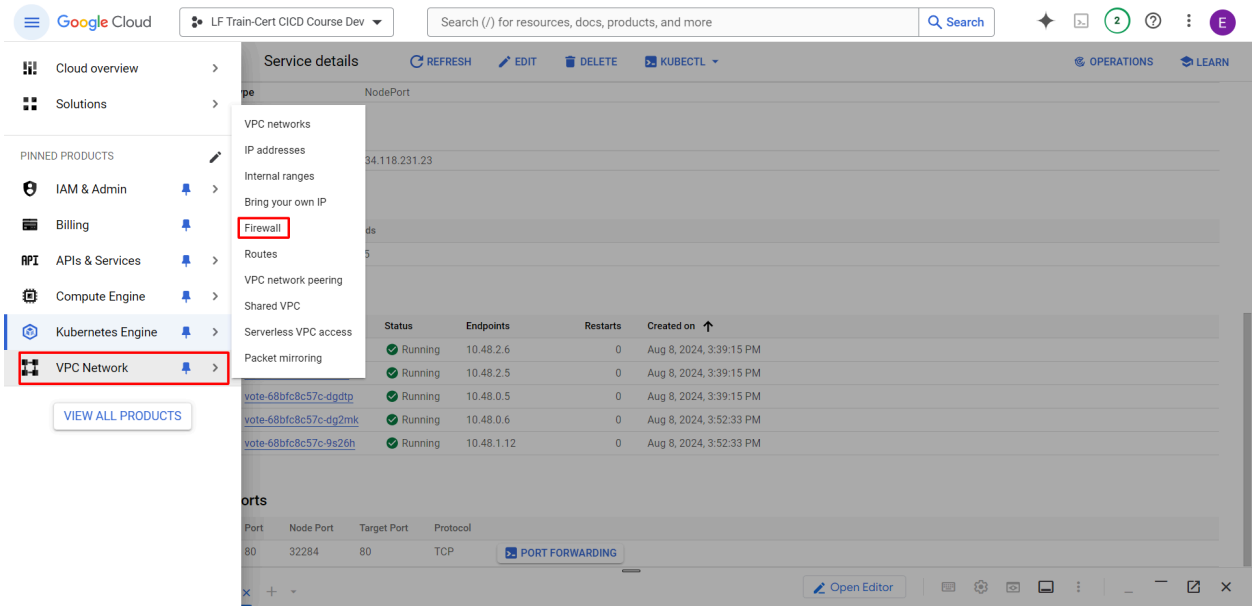


You will need to know this later.

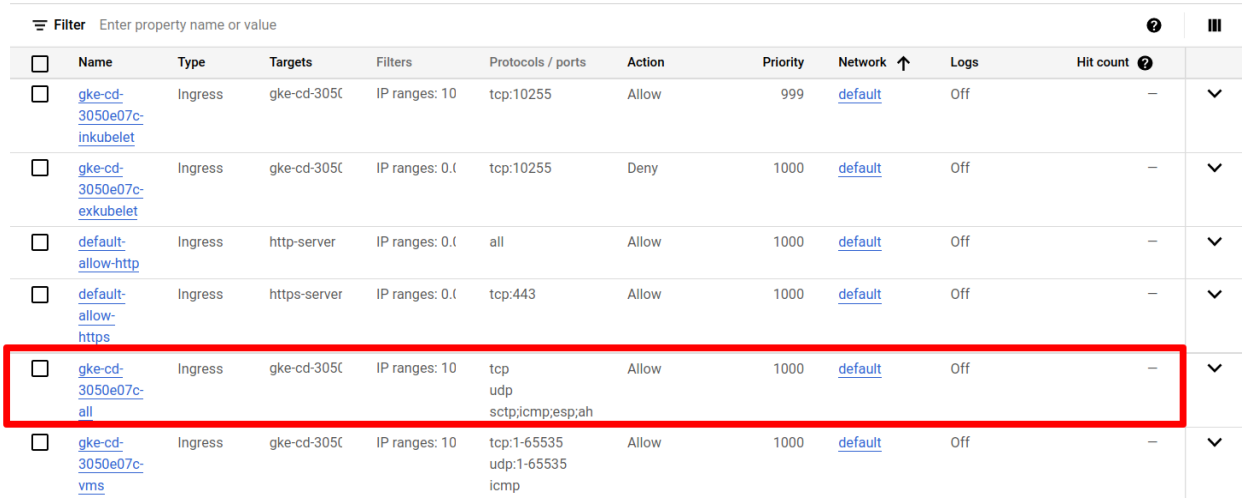
Opening Firewall Rules

You may have already performed the following steps at the time of creating the cluster. If not, open the firewall rules, as follows, in order for you to be able to access the services from outside the cluster (e.g. using a browser on your workstation, which is outside the GKE cluster).

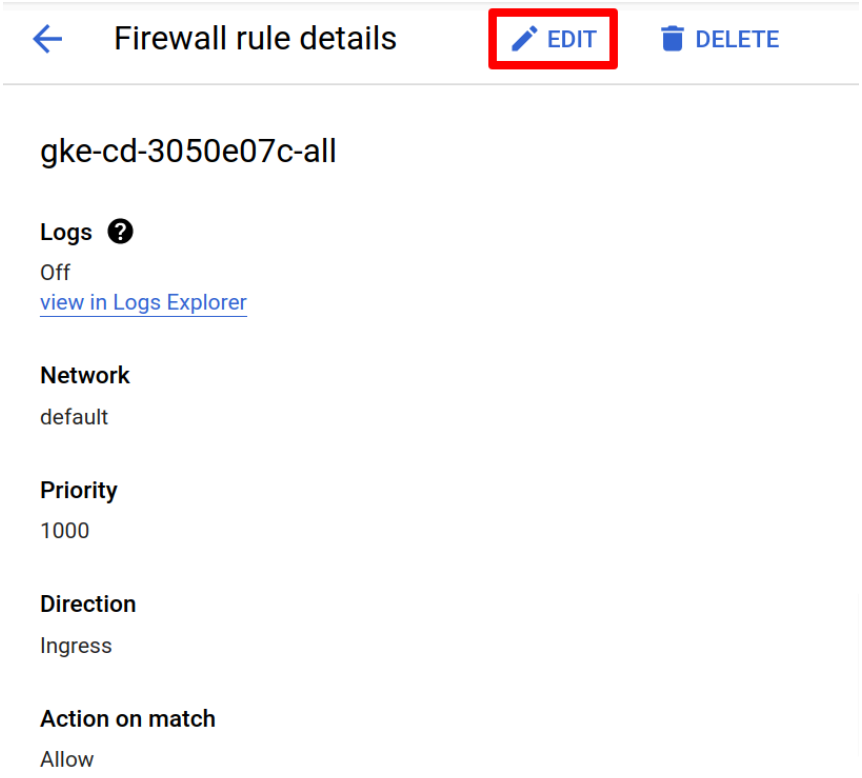
Navigate to the main cloud console menu and **VPC network > Firewall**:



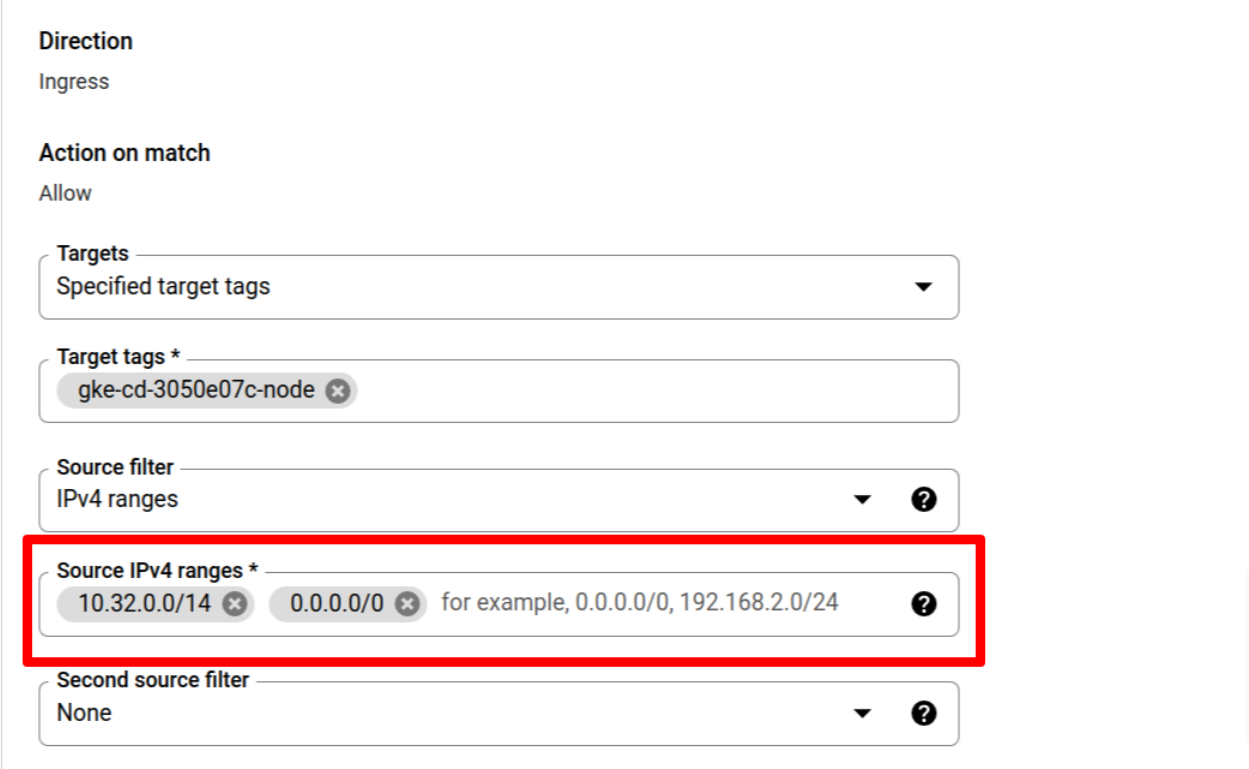
Scroll to the rule that ends in `-all` and click on it:



Click **EDIT**:



Scroll to **Source IPv4 ranges** and enter `0.0.0.0/0`:



Click **Save** at the bottom of the screen.

Find the external IP address of the nodes participating in the Kubernetes cluster.

Alternately, you can find the external IPs of the nodes by using the following command:

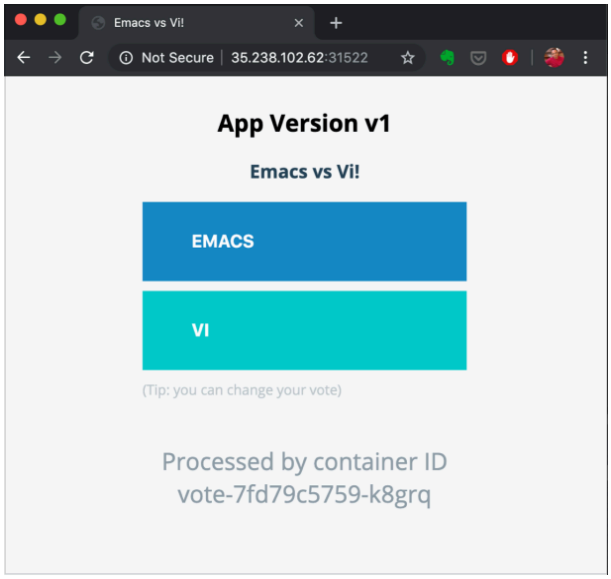
```
kubectl get nodes -o wide
```

Sample output:

```
NAME                                     STATUS   ROLES    AGE   VERSION
INTERNAL-IP   EXTERNAL-IP      OS-IMAGE
KERNEL-VERSION   CONTAINER-RUNTIME

gke-cd-default-pool-4c6b87d8-39cg   Ready    <none>   19h
v1.32.4-gke.1353003   10.138.0.38   34.53.78.232     Container-Optimized OS
from Google   6.6.72+          containerd://1.7.24

gke-cd-default-pool-4c6b87d8-9rbn   Ready    <none>   19h
v1.32.4-gke.1353003   10.138.0.37   34.168.221.251   Container-Optimized OS
```

```
from Google   6.6.72+          containerd://1.7.24

gke-cd-default-pool-4c6b87d8-pwg0   Ready    <none>   19h
v1.32.4-gke.1353003   10.138.0.39   35.185.198.94    Container-Optimized OS
from Google   6.6.72+          containerd://1.7.24
```

Use any of the external IPs and load the application by using NodePort as **`http://IPADDRESS:NODEPORT`**. For example:

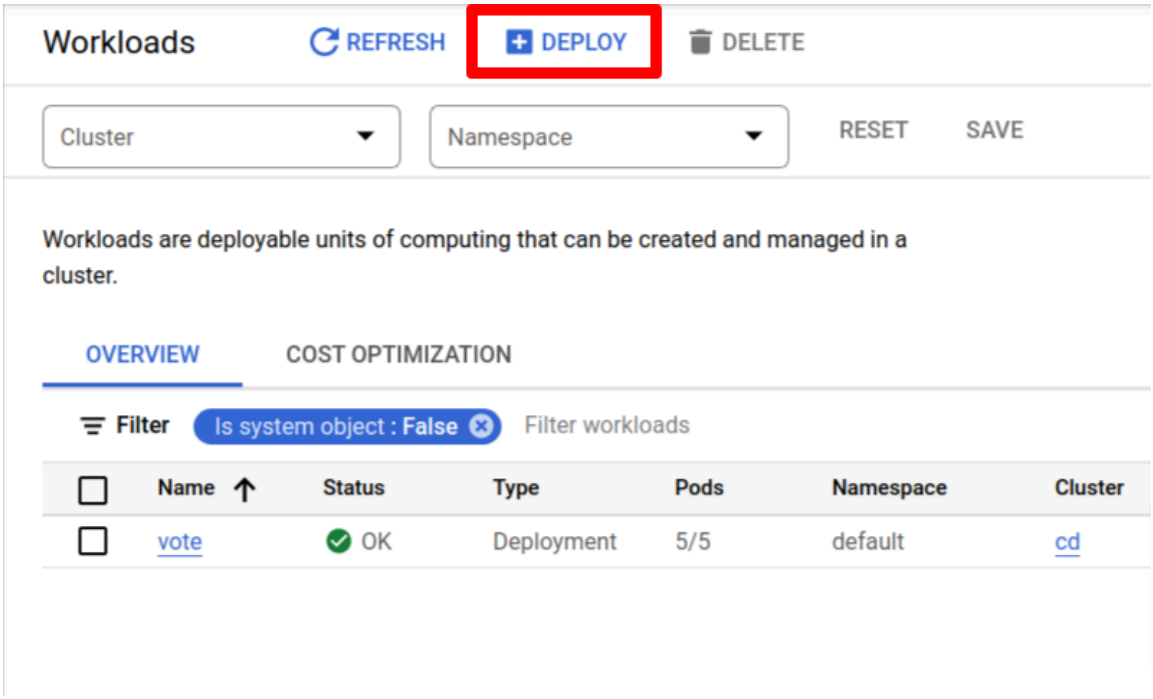


Try reloading the URL a few times and observe the container ID. Do you see the container IDs changing? If yes, the load balancing is in place.

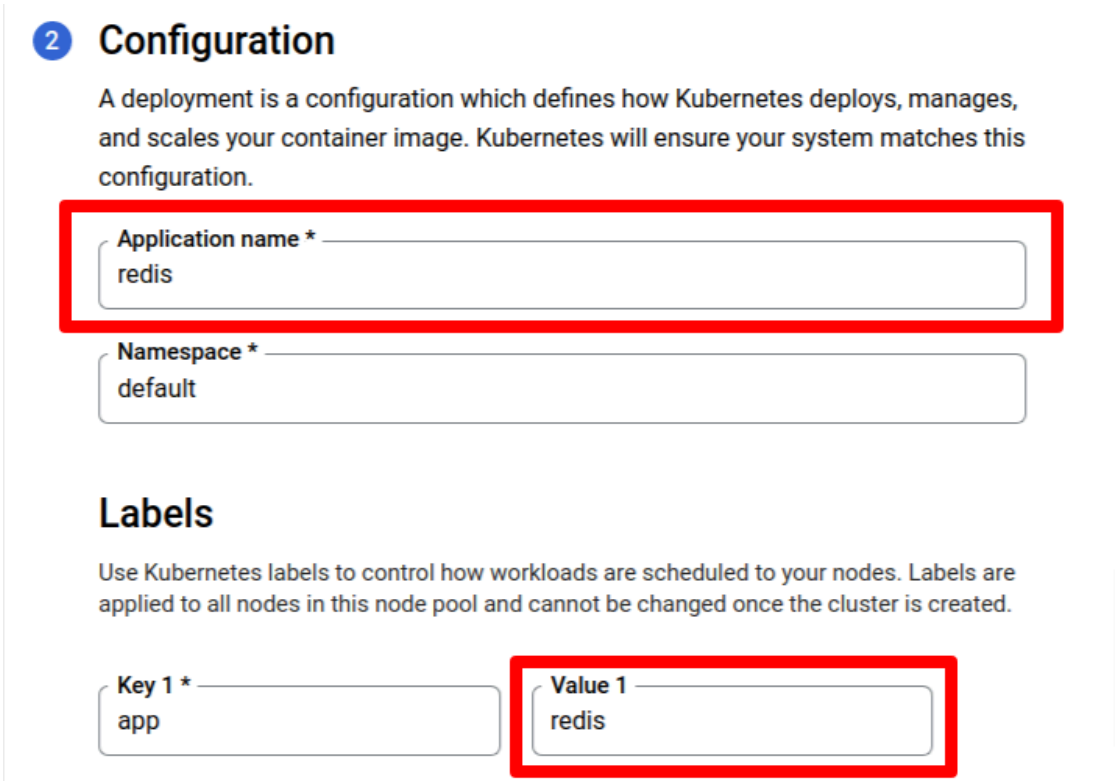
Try clicking an option. You will notice that the app crashes. This is because the **`redis`** backend is not set up. This is what you will do in the next section.

Service Discovery

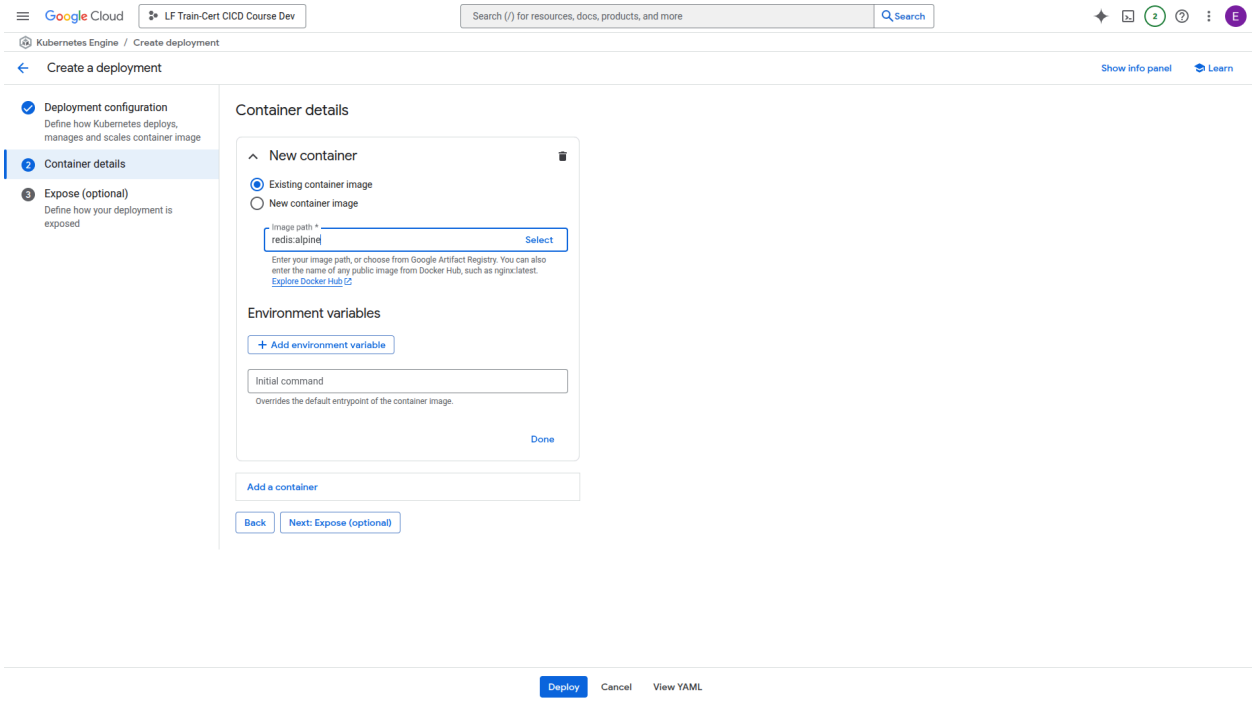
To launch the **`redis`** backend microservice go to **Kubernetes Engine > Workloads** and select the **Deploy** option.



Enter the **Application name** and **Value 1** as `redis`.



Provide the image **`redis:alpine`** and click on **Deploy**.

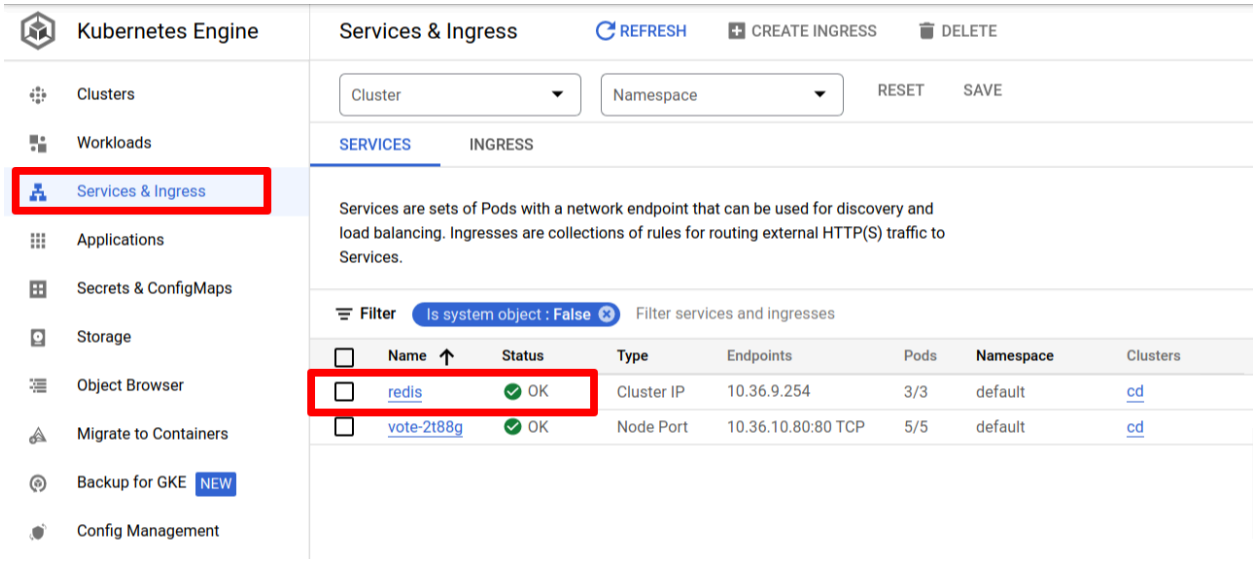


Wait for the **`redis`** deployment to be ready.

Enter the following command into your terminal to expose the **`redis`** application as a service:

```
$ kubectl expose deployment redis --port=6379 --name=redis
service/redis exposed
```

Navigate to **Kubernetes Engine > Services and Ingress**. You will notice the `redis` service is listed:



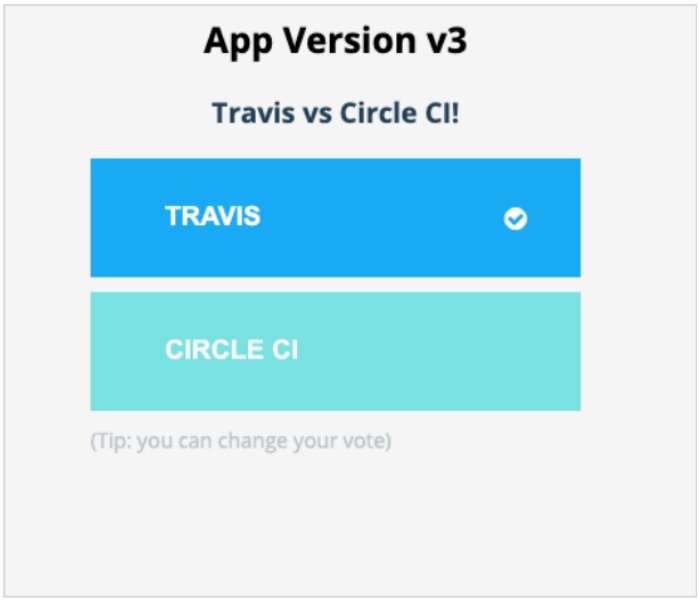
This means you can submit a vote on the vote application and it won't crash.

Run:

```
$ kubectl get node -o wide
```

The output will show your nodes. Under the **`EXTERNAL-IP`** column you will find the public IP's associated with each of your nodes. You can navigate to any of these IP's with the frontend port, which is found at **Kubernetes Engine > Services & Ingress > vote-xxxx > Ports > Node Port**. Enter the following into your web browser to see the frontend and click to vote:

`NODEIPADDRESS:NODEPORT`



As soon as you submit the vote, you should see a tick, which validates the communication between the frontend vote and the backend redis applications.

Summary

In this lab, we set up an account on Google Cloud. We installed the **`kubectl`** command line utility and set it up to interact with our cluster on Google Cloud. We then deployed the voting application to a Kubernetes Cluster on Google Cloud so that we can visit the app from any browser and submit a vote.